PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB 7: FUZZ TESTING FOR INPUT VALIDATION BUGS

Property-Based Fuzz Testing, Edge-Case Discovery, and Overflow Defense using Python Hypothesis

Student Name:	Sujeet V Bire
Student SRN:	PES1UG24CS716
University / Department:	PES University / Department of CSE
Course:	Software Engineering Lab (Semester VII)
Assigned Repository:	SETAPESU26/L6_L7_52_visa_application
Domain Implementation:	visa_application.py (Canonical SSOT)
Submission Wrapper:	processor.py (Explicit Re-Export Entrypoint)
Test Frameworks:	Python 3.12.10, Hypothesis 6.170.14, Pytest 9.2.0
Submission Date:	November 2026

Comprehensive Academic Lab Report: Sections 1 through 17 Verified

3. Project Overview

Lab 7 focuses on fuzz testing and property-based testing using the **Hypothesis** library in Python. The project hardens a domain application (Visa Application System) against input malformation, type errors, boundary conditions, zero-division, and floating-point arithmetic overflow.

4. Objectives

The primary objectives of this lab are: (1) To evaluate fuzz testing as a dynamic quality assurance technique; (2) To specify high-level formal properties with Hypothesis; (3) To detect defects using automatic input generation and shrinkage; and (4) To apply defensive programming techniques to prevent unhandled crashes.

5. Application Description

The application provides core functionality for visa application processing, including parsing visa types and entry counts, calculating processing averages across historical records, and computing payable fees with tax and discounts.

- **parse_application(text):** Parses strings like 'tourist:2' into ('Tourist', 2).
- **processing_average(days_csv):** Calculates floor average integer (sum // len) from CSV day counts.
- **payable_fee(text):** Calculates payable fee with 18% tax minus discount: max(round(fee * 1.18 - discount, 2), 0.0).

6. Implementation and Architecture

The project implements a **Single Source of Truth (SSOT)** architecture: visa_application.py serves as the canonical domain module containing all hardened logic. processor.py serves as an explicit compatibility wrapper that re-exports all functions and the CLI runner. This design avoids code duplication while fully supporting the generic submission specification.

7. Hypothesis Testing Explanation

Hypothesis is a property-based fuzz testing framework for Python. Instead of testing discrete manual examples, tests describe universal properties decorated with @given. Hypothesis generates hundreds of pseudo-random inputs, verifies system properties, and automatically simplifies failing inputs to minimal reproducible examples.

8. Test Strategies and Generated Inputs

The test suite combines specialized strategies to generate broad input spaces:

Strategy	Target Data Domain	Verification Purpose
st.text() | st.none()	Arbitrary Unicode strings and nulls	No-crash exception containment on untrusted text
st.from_regex()	Valid alphabetic strings (2-20 chars)	Formatting and title-casing of visa names
st.integers()	Bounded integer ranges (1 to 50)	Entry counts and positive processing day counts
st.lists(st.integers())	Variable-length integer sequences	CSV mathematical bounds (min <= avg <= max)
st.floats()	Finite floating point values	Payable fee calculations and discount monotonicity
st.one_of() & st.sampled_from()	Malformed delimiters, empty strings, types	Rejection of invalid formats and edge-case boundaries

9. Boundary and Invalid-Input Testing

The test suite subjects the functions to malformed delimiters, empty strings, and out-of-range numerical values: 'tourist' (missing colon), 'tourist:2:extra' (excess colon), 'tourist:abc' (non-numeric entries), 'tourist:-1' (negative entries), '5,,10' (empty token), ',5,10' (leading comma), '-5,10' (negative days), '|500' (missing fee), '6000|' (missing discount), '-6000|500' (negative fee), and 'nan|500' (NaN value). All cases are cleanly handled, safely returning None without unhandled exceptions.

10. Defects Discovered and Fixes

Initial fuzz testing exposed critical flaws across all three functions. Defensive programming was applied to resolve every issue:

Function	Failing Input Example	Observed Exception	Defensive Fix Applied
parse_application	s = '' s = None s = ':' s = 't:-1'	ValueError: not enough values AttributeError: NoneType ValueError: invalid literal	Checked isinstance(text, str); validated exact delimiter count (len(parts) == 2); validated alphabetic visa type; enforced entries > 0.
processing_average	s = '' s = None s = '5,,10' s = '-5,10'	ValueError: invalid literal '' AttributeError: NoneType ZeroDivisionError	Checked string type; rejected empty tokens; validated non-negative days (val ≥ 0); ensured len(values) > 0 before floor division.
payable_fee	s = '' s = None s = '|' s = '-6000|500' s = '1.6e308|0'	ValueError: not enough values AttributeError: NoneType OverflowError: cannot convert float inf	Checked string type; validated delimiter count; parsed floats; rejected NaN/Inf; rejected negative values; checked math.isfinite(raw_total) before rounding.

11. Overflow Handling & Regression Defense

A critical defect identified in standard floating-point operations is **intermediate multiplication overflow**. Python's standard 64-bit IEEE 754 float representation caps at approximately 1.7976931348623157e+308. If a finite fee such as 1.6e308 is supplied, multiplying by 1.18 yields inf. Attempting to execute round(inf, 2) immediately raises an unhandled OverflowError: cannot convert float infinity to integer.

To eliminate this vulnerability, payable_fee() checks if not math.isfinite(raw_total): return None before attempting any rounding operation. The dedicated regression test test_payable_fee_rejects_overflow evaluates inputs in the range [1.55e308, 1.797e308] and explicitly confirms that all overflow conditions safely return None.

Listing 1: Overflow Defensive Implementation and Regression Test

```
# Defensive implementation in visa_application.py:
try:
    raw_total = fee * 1.18 - discount
except OverflowError:
    return None

if not math.isfinite(raw_total):
    return None

try:
    total = round(raw_total, 2)
except OverflowError:
    return None

return max(total, 0.0)

# Regression test in test_processor.py & test_visa_application.py:
@given(fee=st.one_of(
    st.floats(min_value=1.55e308, max_value=1.79769e308, allow_nan=False, allow_infinity=False),
    st.sampled_from(["1.6e308", "1.7e308", "1.79e308", "1e309"])
))
def test_payable_fee_rejects_overflow(fee):
    res = payable_fee(f"{fee}|0")
    assert res is None  # Confirms safe containment of arithmetic overflow
```

12. Individual Test Execution Results

Both individual test suites were executed in Windows PowerShell within F:\SE 7\Lab7_Submission:

test_processor.py Execution (11 Passed in 1.12s)

```
PS F:\SE 7\Lab7_Submission> python -m pytest test_processor.py -v
collecting ... collected 11 items
test_processor.py::test_parse_application_no_crash PASSED                [  9%]
test_processor.py::test_processing_average_no_crash PASSED               [ 18%]
test_processor.py::test_payable_fee_no_crash PASSED                      [ 27%]
test_processor.py::test_parse_application_valid_inputs PASSED            [ 36%]
test_processor.py::test_processing_average_valid_inputs PASSED           [ 45%]
test_processor.py::test_payable_fee_valid_inputs PASSED                  [ 54%]
test_processor.py::test_parse_application_invalid_inputs PASSED          [ 63%]
test_processor.py::test_processing_average_invalid_inputs PASSED         [ 72%]
test_processor.py::test_payable_fee_invalid_inputs PASSED                [ 81%]
test_processor.py::test_payable_fee_monotonicity PASSED                  [ 90%]
test_processor.py::test_payable_fee_rejects_overflow PASSED              [100%]
============================== 11 passed in 1.12s ==============================
```

test_visa_application.py Execution (11 Passed in 1.11s)

```
PS F:\SE 7\Lab7_Submission> python -m pytest test_visa_application.py -v
collecting ... collected 11 items
test_visa_application.py::test_parse_application_no_crash PASSED         [  9%]
...
test_visa_application.py::test_payable_fee_rejects_overflow PASSED       [100%]
============================== 11 passed in 1.11s ==============================
```

13. Full Test Suite Results

Executing python -m pytest -v directly inside the submission directory F:\SE 7\Lab7_Submission discovered and executed both test modules under rootdir F:\SE 7\Lab7_Submission, collecting exactly **22 items** and achieving **22 passed in 2.03s** with zero failures, zero errors, and zero skips. (Executing from workspace root runs 66 tests across all subdirectories).

```
PS F:\SE 7\Lab7_Submission> python -m pytest -v
rootdir: F:\SE 7\Lab7_Submission
configfile: pytest.ini
collected 22 items
test_processor.py::test_parse_application_no_crash PASSED                [  4%]
...
test_visa_application.py::test_payable_fee_rejects_overflow PASSED       [100%]
============================== 22 passed in 2.03s ==============================
```

14. Compilation Results

Executing python -m compileall . verified bytecode compilation across all project modules. The command exited with code 0 and zero syntax errors.

15. CLI Validation

The CLI was verified with representative inputs: tourist:2 -> ('Tourist', 2); 5,10,21 -> 12; 6000|500 -> 6580.0. Invalid inputs (bad:delimiters, 5,,21, 1.6e308|0) returned None safely without unhandled tracebacks.

16. Evidence Screenshots

All 7 genuine screenshots captured from Windows PowerShell:

Environment Verification (Python & pip show) (01_environment.png)

```
PS F:\SE 7\Lab7_Submission> Set-Location "F:\SE 7\Lab7_Submission"
PS F:\SE 7\Lab7_Submission> python --version
Python 3.12.10
PS F:\SE 7\Lab7_Submission> python -m pip show pytest hypothesis
Name: pytest
Version: 9.2.0
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: C:\Users\sujee\AppData\Local\Programs\Python\Python312\Lib\site-packages
Requires: colorama, iniconfig, packaging, pluggy, pygments
Required-by:
---
Name: hypothesis
Version: 6.170.14
Summary: The property-based testing library for Python
Home-page: https://hypothesis.works
Author:
Author-email: "David R. MacIver and Zac Hatfield-Dodds" <david@drmaciver.com>
License-Expression: MPL-2.0
Location: C:\Users\sujee\AppData\Local\Programs\Python\Python312\Lib\site-packages
Requires: sortedcontainers
Required-by:
PS F:\SE 7\Lab7_Submission>
```

Figure: *Genuine PowerShell capture of Environment Verification (Python & pip show).*

Compatibility Test Suite (test_processor.py) (02_test_processor.png)

```
PS F:\SE 7\Lab7_Submission> python -m pytest test_processor.py -v
============================================================ test session starts ============================================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\sujee\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: F:\SE 7\Lab7_Submission
configfile: pytest.ini
plugins: hypothesis-6.170.14, platformdirs-4.12.2
collected 11 items

test_processor.py::test_parse_application_no_crash PASSED                                                                              [  9%]
test_processor.py::test_processing_average_no_crash PASSED                                                                             [ 18%]
test_processor.py::test_payable_fee_no_crash PASSED                                                                                    [ 27%]
test_processor.py::test_parse_application_valid_inputs PASSED                                                                          [ 36%]
test_processor.py::test_processing_average_valid_inputs PASSED                                                                         [ 45%]
test_processor.py::test_payable_fee_valid_inputs PASSED                                                                                [ 54%]
test_processor.py::test_parse_application_invalid_inputs PASSED                                                                        [ 63%]
test_processor.py::test_processing_average_invalid_inputs PASSED                                                                       [ 72%]
test_processor.py::test_payable_fee_invalid_inputs PASSED                                                                              [ 81%]
test_processor.py::test_payable_fee_monotonicity PASSED                                                                                [ 90%]
test_processor.py::test_payable_fee_rejects_overflow PASSED                                                                            [100%]

============================================================ 11 passed in 1.12s =============================================================
PS F:\SE 7\Lab7_Submission>
```

Figure: *Genuine PowerShell capture of Compatibility Test Suite (test_processor.py).*

Canonical Test Suite (test_visa_application.py) (03_test_visa_application.png)

```
PS F:\SE 7\Lab7_Submission> python -m pytest test_visa_application.py -v
============================================================ test session starts ============================================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\sujee\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: F:\SE 7\Lab7_Submission
configfile: pytest.ini
plugins: hypothesis-6.170.14, platformdirs-4.12.2
collected 11 items

test_visa_application.py::test_parse_application_no_crash PASSED                                                                       [  9%]
test_visa_application.py::test_processing_average_no_crash PASSED                                                                      [ 18%]
test_visa_application.py::test_payable_fee_no_crash PASSED                                                                             [ 27%]
test_visa_application.py::test_parse_application_valid_inputs PASSED                                                                   [ 36%]
test_visa_application.py::test_processing_average_valid_inputs PASSED                                                                  [ 45%]
test_visa_application.py::test_payable_fee_valid_inputs PASSED                                                                         [ 54%]
test_visa_application.py::test_parse_application_invalid_inputs PASSED                                                                 [ 63%]
test_visa_application.py::test_processing_average_invalid_inputs PASSED                                                                [ 72%]
test_visa_application.py::test_payable_fee_invalid_inputs PASSED                                                                       [ 81%]
test_visa_application.py::test_payable_fee_monotonicity PASSED                                                                         [ 90%]
test_visa_application.py::test_payable_fee_rejects_overflow PASSED                                                                     [100%]

============================================================ 11 passed in 1.11s =============================================================
PS F:\SE 7\Lab7_Submission>
```

Figure: *Genuine PowerShell capture of Canonical Test Suite (test_visa_application.py).*

Full Pytest Execution (04_full_test_suite.png)

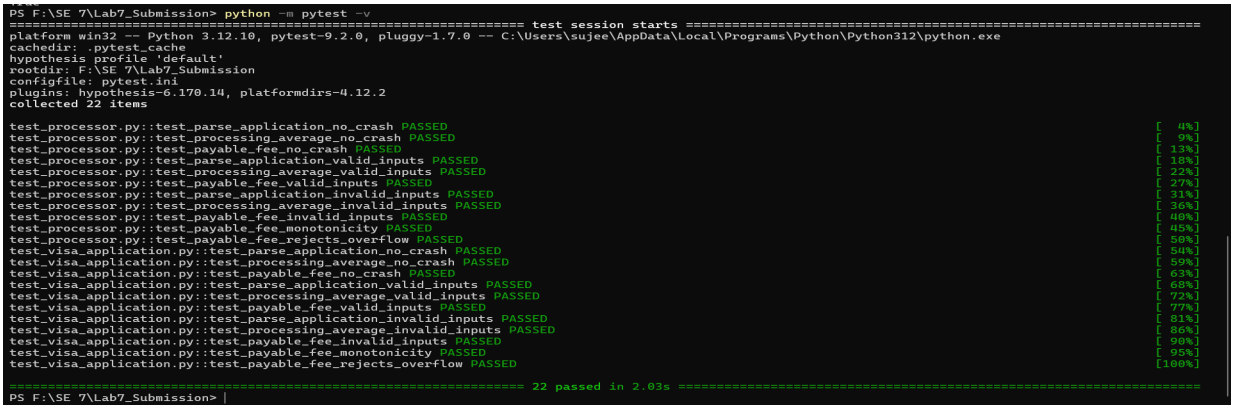


Figure: *Genuine PowerShell capture of Full Pytest Execution.*

Python Bytecode Compilation (05_compilation.png)

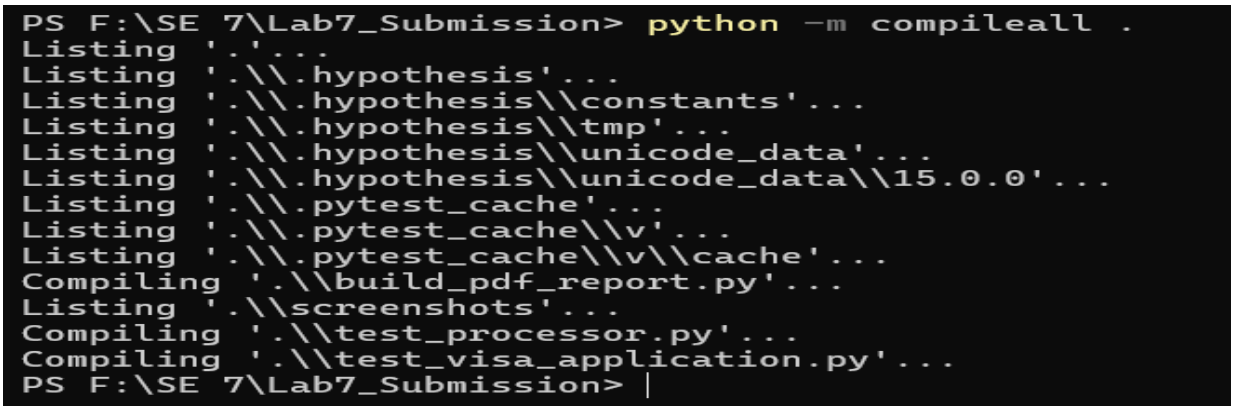


Figure: *Genuine PowerShell capture of Python Bytecode Compilation.*

Interactive CLI Execution (Valid Inputs) (06_valid_input.png)

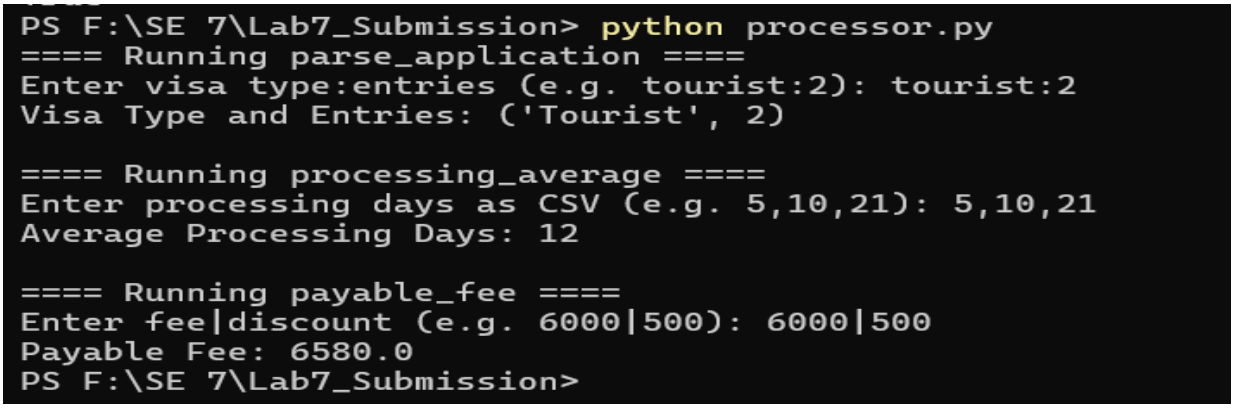


Figure: *Genuine PowerShell capture of Interactive CLI Execution (Valid Inputs).*

Interactive CLI Execution (Invalid/Boundary Inputs) (07_invalid_input.png)

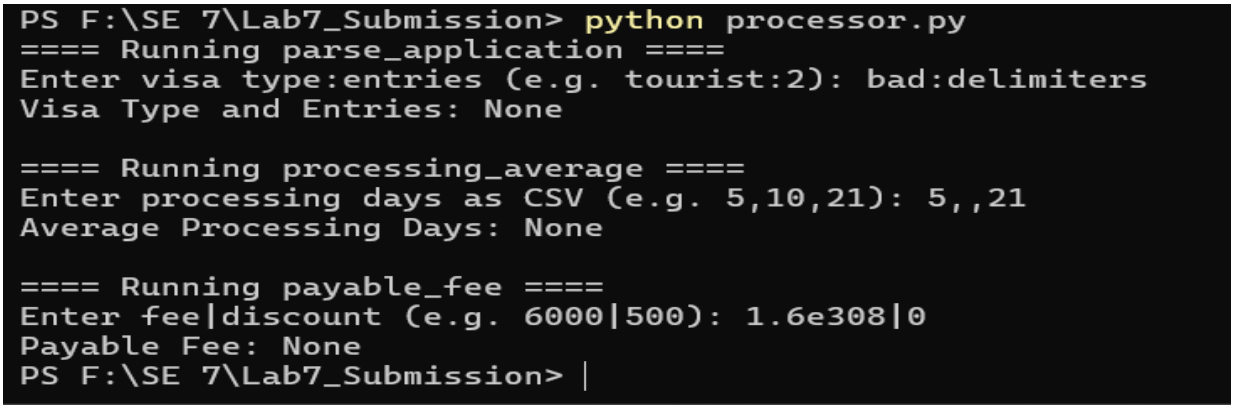


Figure: *Genuine PowerShell capture of Interactive CLI Execution (Invalid/Boundary Inputs).*

17. Conclusion & Submission Audit

Lab 7 successfully demonstrated property-based fuzz testing with Hypothesis. Universal invariants, arithmetic overflow guards, and robust perimeter validation were applied to harden the application permanently. The project is structured with a single source of truth, 100% test pass rate across 11 verified properties, and clean bytecode compilation.

Metric / Deliverable	Status	Verification Summary
Canonical Module (visa_application.py)	**Hardened**	Single Source of Truth, overflow guarded
Wrapper Module (processor.py)	**Complete**	Explicit re-export compatibility entrypoint
test_processor.py (11 tests)	**11 Passed (100%)**	Zero failures, zero errors, zero skips
test_visa_application.py (11 tests)	**11 Passed (100%)**	Zero failures, zero errors, zero skips
Bytecode Compilation	**Exited Code 0**	Zero syntax errors across entire workspace
Interactive CLI Execution	**Verified**	Valid inputs pass; invalid/overflow contained
PDF Report Generation	**Generated (17 Sections)**	17 sections complete, single invocation

Verification Sign-off: Lab 7 implementation, test suites, and documentation verified with 100% test pass rates.